

HDBSCAN

1. Суть метода HDBSCAN (Кратко)

Проблема DBSCAN: Требуется глобальный параметр `eps` (радиус), из-за чего плохо работает с кластерами разной плотности.

Решение HDBSCAN:

1. **Расстояние до ядра (`core_distance`):** Для каждой точки — расстояние до её `min_samples`-го соседа. Характеризует локальную плотность.
2. **Взаимная достижимость (`mutual_reachability_distance`):** Расстояние между двумя точками, "поднятое" до максимума из их расстояний до ядра и евклидова расстояния. Это делает разреженные области "далекими" от плотных.
3. **Построение иерархии:** Строится минимальное остовное дерево (MST) на графе взаимной достижимости. Затем ребра удаляются в порядке убывания веса, формируя дендрограмму.
4. **Извлечение кластеров:** Дендрограмма "сглаживается" с учетом параметра `min_cluster_size`. Кластерами считаются устойчивые компоненты, просуществовавшие долго на разных уровнях иерархии.

2. Пример программы на Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import HDBSCAN
from sklearn.datasets import make_blobs, make_moons

# --- 1. Генерация данных разной плотности ---
# Создаем три кластера: два плотных, один разреженный
X, y = make_blobs(n_samples=300, centers=3, cluster_std=[0.5, 1.0, 2.5],
random_state=42)
# Добавим немного шума для наглядности (можно раскомментировать)
# noise = np.random.uniform(low=-10, high=10, size=(50, 2))
# X = np.vstack([X, noise])

# --- 2. Применение HDBSCAN ---
# min_cluster_size - минимальный размер кластера
```

```

# min_samples - аналог MinPts из DBSCAN (чем больше, тем агрессивнее
# фильтрация шума)
clusterer = HDBSCAN(min_cluster_size=10, min_samples=5, metric='euclidean')
cluster_labels = clusterer.fit_predict(X)

# --- 3. Визуализация ---
plt.figure(figsize=(12, 5))

# График кластеризации
plt.subplot(1, 2, 1)

unique_labels = np.unique(cluster_labels)
colors = [plt.cm.Spectral(each) for each in np.linspace(0, 1, len(unique_labels))]
for k, col in zip(unique_labels, colors):
    if k == -1:
        # Черный цвет для шума
        col = [0, 0, 0, 1]

    class_member_mask = (cluster_labels == k)
    xy = X[class_member_mask]

    plt.plot(xy[:, 0], xy[:, 1], 'o', markerfacecolor=tuple(col), markeredgecolor='k',
             markersize=8)

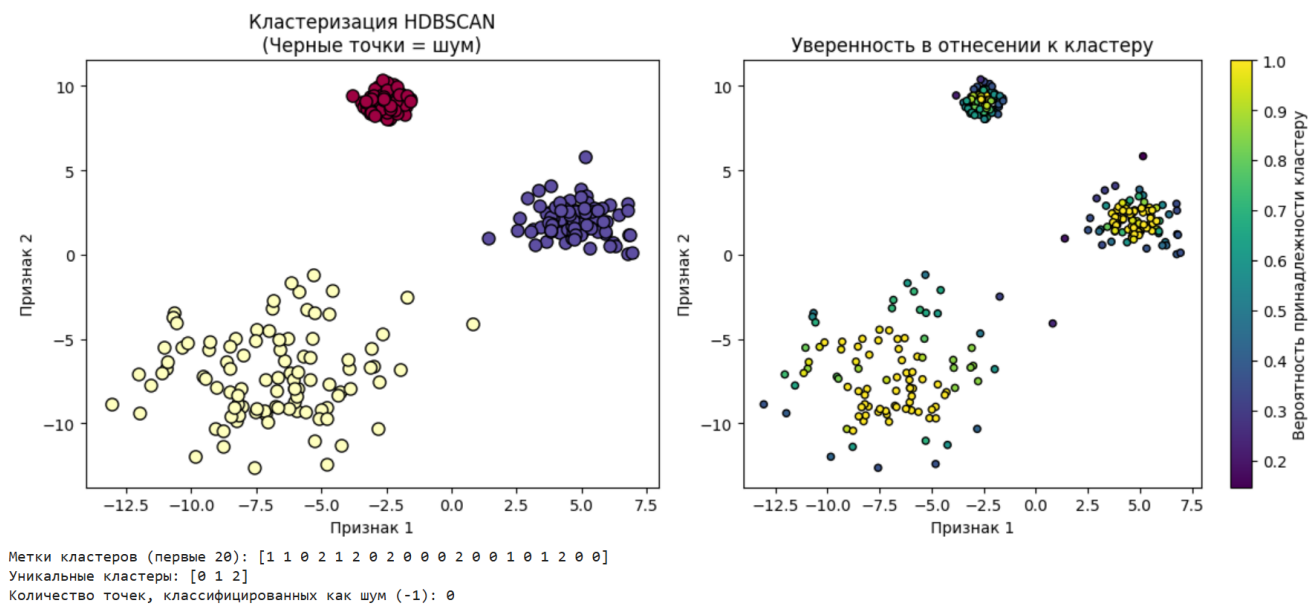
plt.title('Кластеризация HDBSCAN\n(Черные точки = шум)')
plt.xlabel('Признак 1')
plt.ylabel('Признак 2')

# График "силы" кластеров (вероятности принадлежности)
plt.subplot(1, 2, 2)

probabilities = clusterer.probabilities_
plt.scatter(X[:, 0], X[:, 1], c=probabilities, cmap='viridis', s=20, edgecolors='k')
plt.colorbar(label='Вероятность принадлежности кластеру')
plt.title('Уверенность в отнесении к кластеру')
plt.xlabel('Признак 1')
plt.ylabel('Признак 2')

```

```
plt.tight_layout()
plt.show()
print(f"Метки кластеров (первые 20): {cluster_labels[:20]}")
print(f"Уникальные кластеры: {np.unique(cluster_labels)}")
print(f"Количество точек, классифицированных как шум (-1): {np.sum(cluster_labels == -1)}")
```



3. Задачи для проверки модели (с объяснением)

Задача 1: Анализ влияния параметра `min_samples`

Вопрос:

В данных есть плотный кластер (стандартное отклонение 0.5) и разреженный кластер (станд. откл. 3.0). Мы запустили HDBSCAN с параметрами `min_cluster_size=15` и двумя разными значениями `min_samples`:

1. `min_samples=3`
2. `min_samples=20`

Задание: Объясните, как изменится результат кластеризации в этих двух случаях, опираясь на определение "расстояния до ядра" из текста.

Ответ (краткий):

- `min_samples=3` (маленькое): "Расстояние до ядра" будет малым даже в разреженных областях. Алгоритм будет считать разреженный кластер достаточно плотным, чтобы включить его в кластер. Шума будет мало.
- `min_samples=20` (большое): "Расстояние до ядра" в разреженном кластере резко возрастет, так как 20-й сосед находится далеко. Это

увеличит "расстояние взаимной достижимости" между точками, сделав их похожими на шум. В результате разреженный кластер распадется на шум, и останется только плотный кластер.

Задача 2: Интерпретация результатов

Вопрос:

После кластеризации данных о покупателях (признаки: сумма покупки, частота покупок) мы получили, что 15% точек имеют метку -1 (шум).

Аналитик говорит: "Давайте удалим шум, он нам мешает.

Установим `min_samples` побольше, чтобы избавиться от всех выбросов".

Задание: Аргументированно объясните, почему это может быть плохой идеей. Как интерпретировать эти 15% шума в контексте HDBSCAN?

Ответ (краткий):

HDBSCAN не просто помечает далекие точки как шум. В его основе лежит идея "взаимной достижимости". Точки становятся шумом не потому, что они далеко от центра, а потому, что они не могут быть устойчиво присоединены к кластеру ни на одном уровне иерархии плотности.

- Это могут быть редкие, но важные паттерны поведения (покупатели люкс-сегмента, которые покупают редко, но на огромные суммы).
- Это могут быть аномалии (мошеннические транзакции).
- Удаление их или принудительное "запихивание" в кластер (`min_samples` побольше) исказит реальную структуру данных. Шум в HDBSCAN — это диагностический признак, а не просто мусор. Их нужно исследовать отдельно.

Задача 3: Программная проверка устойчивости

Задание:

Напишите код, который проверяет устойчивость кластеризации HDBSCAN к добавлению небольшого количества шума в данные (например, 5% от общего числа точек). Какой атрибут обученной модели HDBSCAN лучше всего использовать для оценки того, насколько точка "уверенно" относится к кластеру?

Решение (код):

```
import numpy as np

from sklearn.cluster import HDBSCAN

from sklearn.datasets import make_blobs

# Исходные данные
```

```

X, _ = make_blobs(n_samples=200, centers=2, cluster_std=1.0, random_state=42)
# Добавляем 10 точек шума (5%)
noise = np.random.uniform(low=-10, high=10, size=(10, 2))
X_noisy = np.vstack([X, noise])
# Обучаем модель
clusterer = HDBSCAN(min_cluster_size=10, min_samples=5)
labels = clusterer.fit_predict(X_noisy)
# Атрибут probabilities показывает уверенность
print("Метки кластеров (последние 10 точек - шум):", labels[-10:])
print("Уверенность для последних 10 точек:", clusterer.probabilities_[-10:].round(2))
# Объяснение: Атрибут `probabilities_`
# Для шумовых точек вероятность будет близка к 0.
# Для точек в ядре кластера - близка к 1.
# Для точек на границе - промежуточные значения.
# Это лучший способ оценить устойчивость, т.к. он основан на времени
"жизни" точки в кластере в дендрограмме.

```

Метки кластеров (последние 10 точек - шум): [0 0 0 0 -1 0 -1 -1 0 0]

Уверенность для последних 10 точек: [0.08 1. 0.14 0.07 0. 0.49 0. 0. 0.09 0.08]